In-Home Assistive Robot

Ethics Statement | November 11, 2026

Team: Gavin Lamphier (Industrial Design), Liam Patel (Mechanical Engineering), Austin Heiss (Computer Science)

Our robot is a small two-wheel unit with a screen. It lets an older adult (the **resident**) take video calls from family caregivers, follows them from room to room, and watches and listens for safety problems such as falls. A caregiver sets what the robot is allowed to do. That means we ask a resident to trust us with a camera and microphone that are often on in their home, and a caregiver to trust us with their account. Below we apply the ACM and NSPE codes to our D0 to D2 design and the Week 9 Schema and Stack.

Privacy summary

Our main rule: the caregiver grants the robot's permissions in the app: **Calls**, **Follow** and **Safety monitoring**. With Safety monitoring on, the robot often records audio and video without the resident pressing anything, so it can notice a fall or a call for help. Because the resident does not start each recording, they must be told up front and always be able to see when it is happening: a light and an on-screen icon show whenever the camera or microphone is on. The resident's **Privacy** button pauses monitoring and calls, and the caregiver is told it was pressed.

How consent is asked. Consent is not implied by use, so it is collected at enrollment instead of at each recording. (1) **Caregiver:** in the app, the caregiver turns each permission on or off, sees what each one records and for how long, and confirms. Permissions can be changed or revoked at any time. (2) **Resident:** before setup, the resident gets an accessible notice, written and read aloud, that says the robot will record audio and video on its own for safety, who sees the recordings, how long they are kept, and how to pause or withdraw. We record the resident's yes, or a legally authorized representative's yes if the resident cannot decide. A caregiver account alone is not that authority. (3) **Calls and Follow:** the robot announces the caller's name before video starts. When the caregiver has granted Calls, the call then connects without the resident pressing Accept. Follow starts when the caregiver turns it on. (4) **Others in the home and test participants:** they are told in advance and sign a separate testing agreement, and the robot avoids rooms the household marks as private. A missed alert or unanswered call is a communication result. We never treat it as medical evidence.

Rows 1 to 4 cover every field in the Week 9 database schema. Rows 7, 9 and 10 come from the caregiver-permission and safety-monitoring model and still need to be added to the D0 to D2 diagrams and the schema. "RAM" means the data is held only in memory and is lost on reboot or when the permission is revoked.

#	Data item	Source	Purpose	Consent	Storage	Access	Retention
1	Caregiver account: caregiver_id, display_name	Identity provider at enrollment	Identify the account; show the caller's name before answering	Enrollment	Server database (SQLite)	Call service; own app; resident sees name	Until unpaired; deleted at project end
2	Robot record: robot_id, last_seen_utc, last_sequence	Robot heartbeat	Route calls; show "online 2 min ago"; reject replayed messages	Enrollment	Server database, latest value only	Call service; paired app	Overwritten each heartbeat
3	Pairing: robot_id, caregiver_id, active	Enrollment	Decide which caregiver controls which robot	Enrollment	Server database	Call service; paired app	Deactivated at once on revoke; row removed once its calls expire
4	Call history: call_id, request_id, status, created_utc, accepted_utc, ended_utc, pair keys	App and robot	Report true call outcome; ignore duplicate retries	Enrollment; caregiver's Calls permission	Server database; RAM copy during call	Call service; the two participants	30 days (proposed), then deleted by a purge job
5	Call control messages: start/end action, event type, cancel reason	App and robot	Start, end and report a call	Calls permission	App and robot RAM	Participants; call service	Discarded after processing (outcome kept in row 4)

#	Data item	Source	Purpose	Consent	Storage	Access	Retention
6	Incoming-call alert: caller name, expiry time, ring sound	Call service to robot	Tell the resident who is calling before video starts	Calls permission	Robot RAM	Resident	At most 60 seconds
7	Permissions: Calls, Follow and Safety-monitoring flags, Privacy-pause state	Caregiver app (pause from robot button)	Control what the robot may record or do	Caregiver grant; resident notice	Server database; copy in robot RAM	Call service; caregiver; robot software	Until changed or revoked; revoke applies at once
8	Live video and audio, both directions	Robot camera/mic; caregiver phone	The conversation	Calls permission	Encrypted in transit (WebRTC); short RAM buffers	The two participants	Calls are not saved; gone when the call ends
9	Safety monitoring: continuous camera and microphone feed, on-robot fall and distress detection	Robot camera, microphone and model	Notice falls or calls for help without the resident acting	Safety-monitoring permission; resident notice	Robot RAM, rolling 60-second buffer (proposed)	Robot software only	Overwritten continuously unless an event is detected
10	Safety event: audio/video clip around the event, event type, time, alert to caregiver	Robot model	Show the caregiver what happened	Safety-monitoring permission; resident notice	Server storage, encrypted (proposed)	Caregiver; call service	30 days (proposed), then deleted by a purge job; caregiver can delete sooner
11	Call setup data (SDP and ICE, including network addresses)	App and robot via signaling server	Connect the video stream	Accepted call	RAM on app, robot and server	Participants	Call only; never logged
12	Person tracking: camera frames, detection boxes, selected person's mask and position	Robot camera and on-robot model	Follow only the resident	Follow permission	Robot RAM, capped at 128 entries	Robot software only	Cleared when Follow stops; no room history kept
13	Safety sensors: depth, clearance, wheel encoder, IMU	Robot sensors	See obstacles; stop safely	Supervised use	Robot RAM	Robot software only	Latest readings only
14	Drive commands and motor feedback; stop reason	Robot planner and motor controller	Move within limits; stop on any fault	Follow permission	Robot RAM; fault code to logs	Robot software; maintainer sees fault code	Each command expires in 100 ms; fault codes as row 17
15	Status screens and recording light: caller, camera/mic on, monitoring on, following on	Robot and app	Show the resident and caregiver what is happening	Enrollment, call	Robot and app RAM	Resident; caregiver	Current state only; stale data is labelled
16	Sign-in: password, one-time sign-in codes, access token, refresh token	Identity provider (Keycloak)	Prove who is calling	Enrollment	Password stays at the identity provider; access token in app RAM; refresh token in iOS Keychain	Identity provider; own app; call service checks token	Refresh token erased on sign-out; token lifetimes **not yet set**
17	Diagnostic log: event ID, component, time, error code, outcome	All components	Explain faults and timing	Enrollment; testing agreement	Log files, four 5 MB files	Maintainer only; no media, names or tokens	7 days (proposed)
18	Setup and safety records: calibration, no-go areas, stopping-distance tests	Team setup and tests	Reproducible, safe configuration	None (not personal)	Config files (JSON)	Team	Kept as engineering records; no names or media
19	Cloud segmentation upload (Meta SAM): image, prompt, returned mask	Team test images	Offline experiment only	Separate upload agreement	Meta's servers (retention unverified)	Meta; team	**Real-person images blocked** until retention is verified

#	Data item	Source	Purpose	Consent	Storage	Access	Retention
20	User-test notes: feedback, errors, help needed	Trials	Check safety and accessibility	Separate testing agreement	Not collected yet	Team	Nothing collected until an approved study schedule
21	Reminder prototype (separate stationary app): reminder text, due time, approval, response, caregiver sharing grant	Resident via app	One-time, non-medical reminder	Resident approves each one; sharing is a separate grant	Prototype SQLite	Resident; caregiver only if granted	Synthetic test data only; no automatic purge yet

Fields we removed or refuse to collect. D2 removed access_token from call request bodies, so tokens never reach the database or logs. We could not justify any of these, so none are collected: diagnosis or medical data, home address, birth date, biometric identity (face recognition), room-by-room location history, or saved call recordings. Safety monitoring saves only clips around a detected event. Depth sensing for safety is kept separate from the camera and microphone.

Deletion as a mechanism. Rows 4, 10 and 17 need a scheduled job that deletes records by age, including SQLite journal files and backups. Size-based log rotation alone is not deletion. RAM data, including the rolling safety buffer, is cleared by reboot, revoke or expiry. **When the project ends**, we revoke every account and robot credential, delete the server database, saved safety clips, logs, exports, backups and the app's Keychain and cache, and keep only reports with no personal data. **Still to build and test:** the purge job, disk encryption and file permissions on the server.

Security summary

1. Default credentials. No shared or default passwords ship with the robot. Each robot gets its own credential, which is kept outside source control. The test fixtures use local-only role labels that cannot sign in remotely. **Not yet verified:** per-robot provisioning, removal of any setup account, and credential rotation.

2. Unauthenticated endpoints. Every HTTP and WebSocket call needs a bearer token from our identity provider (OpenID Connect with PKCE through Keycloak). The server checks the token's issuer, audience, expiry and subject, then checks that an active pairing exists. Only the two participants can exchange call setup messages. All public traffic uses HTTPS/WSS, and video uses encrypted WebRTC. A maintainer can read metadata but cannot turn on the camera, view safety clips or move the robot. Motor commands travel only over a local USB link and are rejected if stale, out of order or from before a reboot. Messages have size limits: 65,536 bytes for SDP, 4,096 bytes for ICE and serial frames, and 128 queued ICE candidates. **Not yet configured:** token lifetimes, refresh-token rotation, revocation delay and **rate limiting**.

3. Secrets in source control. Server and device secrets live in the deployment's OS secret store. The iOS app keeps its refresh token in the Keychain. Tokens are never written to the database or logs. .gitignore excludes .env, key and credential files.

Git history search. We wrote a read-only scanner, tools/audit_ethics.py, that searches for private keys, AWS, GitHub and API tokens, JWTs, passwords in URLs and password assignments. It searches **every commit in local history** plus text inside PDFs and ZIPs. The latest run (commit ff03225, 11 commits, 60 file versions) found **zero real secrets**; its 11 matches were schema placeholders. Nothing needed rotating or removing from history. The fix we did make was the D2 change in the privacy section, which keeps tokens out of request bodies. The audit record lists what it could not check, such as images.

Self-audit question	Answer
Are default credentials removed?	Yes by design: unique per-robot credentials. Deployed configuration not yet available to verify.
Can someone call, watch or move the robot without permission?	No by design: token plus active pairing, and each caregiver-granted permission is checked on the robot. Negative tests still to run.
Are tokens, traffic and request rates limited?	HTTPS and token expiry required; message size limits set. Token lifetimes and rate limits still to set.
Is stored data protected and deleted?	Only the call service can open the database. Encryption, permissions and the purge job still to build and test.
Was the git history searched for secrets?	Yes: all local commits, zero real secrets found.

Code of ethics. ACM 1.6 and 2.9 (respect privacy; build secure systems) are why monitoring keeps only a short rolling buffer and saves clips only around detected events, why access is scoped, and why every permission is revocable and visible to the resident. NSPE I.1 (public safety first) is why the robot does not move until its stopping is qualified. ACM 1.4 (be fair and do not discriminate) is why D2 uses plain labels, large scalable text, VoiceOver and states that never rely on colour alone.

Licensing summary

Our license: team-written code is under the MIT License. Every row below keeps its own terms. Licenses use SPDX identifiers. "Not pinned" means we will fix the version in a lockfile before release. "(build)" marks tools used only to make documents.

Name	Version	License (SPDX)	Type	Obligation on us
Python and standard library	3.12.14 (build); robot runtime not pinned	PSF-2.0	Permissive	Keep license notices
SQLite	Not pinned	blessing (public domain)	Public domain	None for the core
FastAPI; Pydantic	Not pinned	MIT	Permissive	Keep copyright notice
Uvicorn; Authlib	Not pinned	BSD-3-Clause	Permissive	Keep notice; no endorsement claims
Keycloak	Not pinned	Apache-2.0	Permissive	Keep license and NOTICE; mark changes
NumPy	Not pinned	BSD-3-Clause	Permissive	Keep notice; check bundled math libraries
PyTorch / TorchVision	Not pinned	BSD-3-Clause	Permissive	Keep notices; model weights have separate terms
MuJoCo (simulation)	Not pinned	Apache-2.0	Permissive	Keep license and NOTICE; mark changes
Swift compiler and runtime	Not pinned	Apache-2.0 WITH Swift-exception	Permissive	Runtime exception covers compiled app
SwiftUI / Apple SDK	Not pinned	NOASSERTION (Apple terms)	Proprietary	Follow Apple developer agreement
Node.js and browser WebRTC (call test bench)	23.7.0; 24.22.1 (reminder app)	MIT	Permissive	Keep notices; check browser terms
Meta SAM model / hosted API	SAM 3 / 3.1	LicenseRef-SAM; Meta API terms	Custom	Keep SAM terms and acknowledgment
Open Beni robot design (reference)	Current repository	MIT	Permissive	Keep notices; archived items unverified
Detector weights, test videos	Not pinned	NOASSERTION	Unknown	Not redistributed until rights confirmed
Ubuntu packages, ST sensor driver/firmware, optional relay	Not selected	NOASSERTION	Unknown	Inventory licenses once chosen
ReportLab 4.4.9; pypdf 6.10.0 (build)	Pinned	BSD-3-Clause	Permissive	Keep notice if distributed
Pillow (build)	13.0.0	MIT-CMU	Permissive	Keep notice
Tectonic (build)	0.17.0	MIT	Permissive	Keep notices if distributed
LaTeX packages (build)	Not pinned	LPPL-1.3c	**Weak copyleft**	Only applies if we modify a package: rename it and keep LPPL
Poppler (build, PDF checks)	26.05.0	GPL-2.0-or-later	**Copyleft**	Used as a separate program only; not bundled or linked
Times New Roman / Liberation Serif fonts (build)	macOS 5.01 / 2.x	NOASSERTION / OFL-1.1	Proprietary / permissive	Font files are not committed; embedding in PDF allowed
Team diagrams, screen layouts, test fixtures	Repository	MIT (team code)	Team-owned	Confirm any adapted assets

Compatibility and copyleft decisions. MIT is compatible with every permissive row as long as we keep their notices. The two copyleft items, Poppler (GPL) and LaTeX packages (LPPL), are document tools we run separately and never ship or link, so they place no copyleft duty on our code. We will not redistribute SAM, Apple SDK,

weights or videos until rights are confirmed (ACM 1.5). Meta's zero-data-retention option does not cover file uploads, so real-person images stay blocked (privacy row 19).

Attribution list

The same list is in NOTICE.md.

- **Server and sign-in:** FastAPI by Sebastián Ramírez and contributors; Uvicorn by Encode OSS; Pydantic by Pydantic Services; Keycloak contributors; Authlib by Hsiaoming Yang; SQLite authors (public domain).
- **Robot, app and computation:** Python Software Foundation; NumPy developers; PyTorch contributors; TorchVision by Soumith Chintala and contributors; MuJoCo by Google DeepMind; Swift authors; Apple SDK agreements; Node.js contributors.
- **Models, hardware and reference designs:** Meta SAM (license), SAM output docs and hosted API terms; Open Beni by Mondo Robotics (MIT); Ubuntu package policy; ST VL53L9CX depth sensor documentation.
- **Document tools and fonts:** ReportLab; Pillow contributors; pypdf by Mathieu Fenniak and contributors; Tectonic contributors; LaTeX Project; Poppler developers; Times New Roman by Monotype; Liberation fonts by Red Hat (OFL-1.1). Our diagrams use built-in draw.io and LaTeX shapes. We use no stock icons, photos or music.
- **Adapted code:** our local call package (0.1.0) imports and adapts the team's earlier shared-call-help-authority-lab (0.1.0) prototype. The changes are recorded in response-adaptation.diff and in the source record.
- **Datasets:** none from residents; test media is synthetic. Public videos and checkpoints need provenance records before redistribution.

AI-use disclosure

> *"Generative AI tools may be used for drafting and code assistance but not for the design decisions or written analysis that are assessed in this course."*

I am Austin Heiss, the team's documentation lead, and this is my account of how AI was used on our project. I sorted each use with the class's two-column framework: "generally OK for assistance" or "not a substitute for" our own assessed analysis. I recorded every member's uses on November 11.

Tool	Used by	Task	Column	How verified
Codex; Claude	Austin Heiss (me)	Drafting documentation, prototype code, research summaries, formalizing our team's ideas	Generally OK	I review and hand-edit every output.
ChatGPT	Liam Patel	Drafting documentation, prototyping, research, formalizing ideas	Generally OK	Liam reviews and hand-edits every output. Liam completes this disclosure in a separate submission for their own class.
None	Gavin Lamphier	Industrial design work and research	Not applicable	Gavin told me they used no AI for these tasks. Gavin completes this disclosure in a separate submission for their own class.
Codex; Claude	Austin Heiss (me)	Wording and formatting of this statement, the constraints essay and the Week 9 document	Not a substitute for (written analysis)	The positions come from our team's D0 to D2 decisions; AI only restated them. I reviewed every section against those designs before signing.

Section owners. I own this document, the software and security. Gavin (consent and accessibility) and Liam (hardware and safety rows) had to complete this assignment separately for their own classes, so their section checks and signatures are in those submissions, not this one. See also the Constraints Essay.

By signing, I confirm that the architecture, data model, priorities and design rationale are our team's own, and that this account of AI use is accurate.

Gavin Lamphier and Liam Patel - Not signed here: each completed this assignment separately for their own class.
Austin Heiss - Signature: Austin Heiss, Date: November 11, 2026